

Checking if a String is a Palindrome

A foundational algorithm in string processing — exploring symmetry, comparison logic, and Python implementation.

DIPANSHU

DIVISION: AL22

PRN: 202501110149

BATCH: A22



What is a Palindrome?

A **palindrome** is a word, number, phrase, or string that reads **the same forward and backward**. The term comes from the Greek words *palin* (again) and *dromos* (direction). Palindromes are a classic example of symmetry in language and data — and they show up more often than you might think in programming, biology, and cryptography.

Classic Examples

- **madam** — reads the same both ways
- **racecar** — a favorite in coding examples
- **level** — simple and elegant
- **121** — numeric palindrome
- **kayak** — a word with real-world meaning

The Core Rule

The defining property of a palindrome is straightforward: if you write the string in reverse, it must be **identical** to the original. This makes palindrome checking one of the most intuitive entry points into string manipulation algorithms.

📌 Case matters in raw comparison — "Madam" ≠ "madaM" — but we can normalize strings before checking to handle this gracefully.

How It Works: The Core Logic

At its heart, palindrome checking is a **three-step process**: accept input, reverse it, and compare. If the reversed version matches the original exactly, you have a palindrome. If not, you don't. This simplicity is what makes it such a great teaching example for beginners learning about strings and algorithms.



This approach is sometimes called the "**naive**" or "**direct comparison**" method. It's not the most memory-efficient technique for extremely large datasets (since it creates a copy of the string), but for learning purposes and most practical applications, it's perfectly suitable and very easy to understand.

Input

Accept any string from the user — a word, number, or phrase.

Reverse

Create a reversed copy of the string using slicing or a loop.

Compare

Check if `original == reversed`. True means palindrome, false means it isn't.

The Algorithm Step by Step

Before writing any code, it's important to understand the algorithm in plain language. An algorithm is a step-by-step recipe for solving a problem. Here is the complete algorithm for palindrome checking, written in structured pseudocode that maps directly to Python:

01	02	03
Start	Input the String	Normalize the String
Begin the program execution.	Read a string from the user and store it in a variable (e.g., <code>s</code>).	Convert the string to lowercase using <code>s.lower()</code> to eliminate case-sensitivity issues. This ensures "Madam" and "madam" are treated the same.
04	05	06
Reverse the String	Compare Original and Reversed	Output the Result
Create a reversed version of the string. In Python, this is done elegantly with slice notation: <code>s[::-1]</code> .	Check if the normalized original string equals the reversed string using the equality operator <code>==</code> .	If equal → print " Palindrome ". If not equal → print " Not a Palindrome ".
07		
Stop		
End the program.		

Python Program

Here is the complete, working Python program for checking palindromes. It's compact, readable, and demonstrates several key Python features including function definition, string methods, and slice notation.

The Complete Code

```
def is_palindrome(s):  
    s = s.lower()          # Normalize case  
    if s == s[::-1]:      # Compare with reverse  
        return "Palindrome"  
    else:  
        return "Not a Palindrome"  
  
# Taking input from user  
string = input("Enter a string: ")  
print(is_palindrome(string))
```

How to Run It

1. Open any Python environment (IDLE, VS Code, Jupyter, or an online interpreter)
2. Type or paste the code above
3. Run the program
4. When prompted, type a word like `madam` or `hello`
5. Press Enter to see the result

Sample Runs

Input: `racecar` → **Output:** `Palindrome`

Input: `python` → **Output:** `Not a Palindrome`

Input: `Noon` → **Output:** `Palindrome` (case handled!)

Code Explanation: Line by Line

Understanding each part of the code is just as important as writing it. Let's break down every line so you know exactly what's happening under the hood.

```
def is_palindrome(s):
```

This line **defines a function** named `is_palindrome` that takes one parameter `s` (the string to check). Functions let you package reusable logic that can be called multiple times with different inputs.

```
s = s.lower()
```

This converts the entire string to **lowercase**. Without this step, "Madam" would not equal "madaM" and the check would fail incorrectly. Normalizing the string ensures case differences don't interfere with the comparison.

```
s[::-1]
```

This is Python's **slice notation for reversing a string**. The syntax `[start:stop:step]` with a step of `-1` walks through the string backward, producing a reversed copy. It's one of Python's most elegant one-liners.

```
if s == s[::-1]:
```

The **equality operator** `==` compares the original (now lowercase) string with its reversed version. If every character matches in order, the condition is `True` and we have a palindrome.

```
return "Palindrome" / return  
"Not a Palindrome"
```

The **return statement** sends the result back to wherever the function was called. Using `return` instead of `print` inside the function makes it more flexible — the caller can decide what to do with the result.

Manual Walkthrough: Two Examples

Let's trace through the algorithm manually with two different inputs — one that **is** a palindrome and one that **isn't**. Walking through examples by hand is an essential skill for understanding how algorithms behave step by step.

✓ Example 1: "madam"

Step 1 — Input: `s = "madam"`

Step 2 — Normalize: `s.lower()` → "madam" (already lowercase)

Step 3 — Reverse: `s[::-1]` → "madam"

Step 4 — Compare: `"madam" == "madam"` → True ✓

Step 5 — Output: `Palindrome`

📄 Every character matches its mirror position — $m \leftrightarrow m$, $a \leftrightarrow a$, $d \leftrightarrow d$. Perfect symmetry!

✗ Example 2: "hello"

Step 1 — Input: `s = "hello"`

Step 2 — Normalize: `s.lower()` → "hello" (already lowercase)

Step 3 — Reverse: `s[::-1]` → "olleh"

Step 4 — Compare: `"hello" == "olleh"` → False ✗

Step 5 — Output: `Not a Palindrome`

📄 The first letter 'h' does not match the last letter 'o', so symmetry is broken immediately.

Running the Code: Live Example

Here's what happens when you actually run the Python function with different inputs. These examples show how the function behaves in a real Python session, including edge cases.



Test Case: "level"

```
>>>
print(is_palindrome("level"))
Palindrome
```

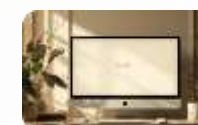
The string "level" is a perfect palindrome. The function normalizes it (no change needed), reverses it to "level", compares, and returns the success message.



Test Case: "NoOn" (Mixed Case)

```
>>>
print(is_palindrome("NoOn"))
Palindrome
```

Even with mixed capitalization, the `.lower()` call normalizes "NoOn" to "noon" before comparing. This is why normalization is critical — without it, this would incorrectly return "Not a Palindrome".



Test Case: "algorithm"

```
>>>
print(is_palindrome("algorithm"))
Not a Palindrome
```

The string "algorithm" reversed is "mhtirogla" — completely different. The comparison fails and the function correctly identifies it as not a palindrome.

Advantages & Disadvantages

Every algorithm has trade-offs. Understanding the strengths and limitations of the palindrome-checking approach helps you decide when and how to use it — and when you might need a more sophisticated solution.

✓ Advantages

- **Simple and Easy to Implement** — The entire logic fits in fewer than 10 lines of Python, making it ideal for beginners.
- **Fast for Small Strings** — For typical word-length inputs, the comparison is nearly instantaneous with $O(n)$ time complexity.
- **Excellent Teaching Tool** — Introduces string slicing, functions, conditionals, and comparison operators in one cohesive example.
- **Readable Code** — Python's `s[::-1]` syntax is intuitive and self-documenting.
- **Foundation for Advanced Topics** — Understanding this builds toward more complex string algorithms like longest palindromic substring.

⚠️ Disadvantages

- **Case Sensitivity (if not handled)** — Without `.lower()`, "Madam" would fail the check. Always normalize input first.
- **Spaces and Special Characters** — A phrase like "A man a plan a canal Panama" contains spaces that break the simple comparison. These must be stripped before checking.
- **Memory Usage** — The method creates a full reversed copy of the string, doubling memory usage. For very large strings, a two-pointer approach (comparing characters from both ends) is more memory-efficient.
- **Not Optimal for Massive Data** — In production systems processing millions of characters, more advanced algorithms (like Manacher's algorithm) are preferred.

Real-Life Applications & Conclusion

Palindrome checking isn't just a classroom exercise — it appears in surprising places across science, technology, and entertainment. Here's where this concept shows up in the real world:



Word Games & Puzzles

Crossword puzzles, Scrabble, and word games frequently use palindromes as clues or special scoring patterns. Many puzzle apps include palindrome detection as a core feature.



Data Validation

Some data validation systems check for palindromic patterns in identifiers, checksums, or encoded data to detect errors or ensure data integrity during transmission.



DNA Sequence Analysis

In bioinformatics, palindromic sequences in DNA (called **inverted repeats**) are biologically significant. They often indicate restriction enzyme cut sites and are critical in genetic research.



Cryptography

Certain cryptographic algorithms and hash functions analyze symmetric patterns in data. Palindrome detection can be part of pattern analysis in security and encoding systems.

Key Takeaways



A **palindrome** reads the same forward and backward — a simple but powerful concept in string processing.



Python's `s[::-1]` and `.lower()` make implementation elegant and concise.



The algorithm: **normalize** → **reverse** → **compare**. Three steps, endless applications.



This concept bridges beginner programming to advanced topics in **bioinformatics**, **cryptography**, and **algorithm design**.